FPGA NN Lab

Task 4: GTSRB Image Classification

Chair of Dependable Nano Computing, Karlsruhe Institute of Technology
Instructors: Vincent Meyers, Johannes Reibold

June 30, 2026

1 Introduction

In this laboratory task, you will create a hardware accelerator for the German Traffic Sign Recognition Benchmark (GTSRB) image classifier on the PYNQ-Z2 board.

Unlike the previous task where a pre-trained model, the build script, and hardware-to-software pipelines were fully prepared, in this task you will design and train a quantized CNN using Brevitas, configure the **FINN+** compiler, build the FPGA bitstream overlay, and write the evaluation notebook yourself.

Your objectives are:

1. Implement and train a quantized CNN with using Brevitas, and export the trained model to QONNX.
2. Merge the pre-processing (normalization) and post-processing (TopK argmax) transformations directly into the ONNX graph so they are executed in hardware.
3. Configure a `build.yaml` and run the FINN+ compiler to compile the merged model.
4. Copy the deployment package to the PYNQ-Z2 board.
5. Implement an evaluation notebook using only NumPy to compute classification accuracy, latency, and throughput (FPS) in batches of 1000.

You are provided with `dataset.py`, a reusable dataset script containing the pure NumPy/PIL implementation of `GTSRBDataset` along with PyTorch wrappers used for training.

2 Task 1: Quantized Model Design, Training, and Export

In this task, you will implement a convolutional neural network (CNN) for GTSRB classification and train it using Brevitas.

2.1 Model

You can choose your own architecture for the model. You can make use of Maxpooling to reduce computational overhead. Note that the feature map size should be a multiple of the pooling kernel size. Make sure to use low bit widths for an efficient model, but keep the input to 8 bits.

2.2 Training

Using the PyTorch datasets provided by `dataset.py`, write a training script to train the model. Monitor validation accuracy and save the best checkpoint. Aim for a validation accuracy of at least 90%.

2.3 Exporting to QONNX

Once the model is trained, use Brevitas's export utility to export the best model checkpoint to QONNX format.

```
from brevitas.export import export_qonnx

model.eval().cpu()
export_qonnx(model, torch.randn(1, 3, 32, 32), export_path="gtsrb_cnn.onnx")
```

Verify the generated ONNX model using Netron (https://netron.app) to ensure the layers and quantized data types are correctly exported.

3 Task 2: Pre-processing & Post-processing Graph Merging

To achieve high hardware throughput, all computations should run in hardware. However, PyTorch models expect standardized float inputs and output raw logits. If done on the CPU of the board (ARM PS), float conversion, normalization (division by 255, mean/std subtraction), and finding the class prediction (argmax) will bottleneck your execution.

Hint: You can define a custom FINN build step (e.g., in a helper file `transform_input.py`) to inject these transformations directly into the graph:

1. **Normalization**: Create a simple PyTorch `Module` representing the normalization function ($x/255.0$ and mean/std subtraction). Export it using Brevitas's `export_qonnx` with a float32 dummy input. Merge it with your trained classifier model using the QONNX transformation `MergeONNXModels`.

2. **UINT8 Input Annotation**: Annotate the new input tensor of the merged model as `DataType["UINT8"]` using `model.set_tensor_datatype`. This allows the hardware DMA to ingest raw 8-bit image pixels directly. FINN's streamlining pass (`step_streamline`) will automatically absorb the division and normalization operations into the thresholds of the first quantization layer!

3. **Post-processing (Argmax)**: Append an ONNX `InsertTopK(k=1)` node at the output. This performs the argmax in hardware and returns the predicted class index directly, avoiding logit transfers and CPU-based class index calculations.

Specify the custom step in the `steps` list of your `build.yaml` configuration immediately after `step_qonnx_to_finn`. Ensure the overall build targets `target_fps: 10000`.

3.1 Listing Custom Build Steps in YAML

In FINN+, you can override the sequence of compilation steps by specifying a list of strings under the `steps` key in your `build.yaml`. Any custom step defined in an external Python module can be listed using the dot notation (`module_name.function_name`).

Your custom step function must adhere to the standard FINN build step signature:

```
def step_name(model: ModelWrapper, cfg: DataflowBuildConfig) -> ModelWrapper:
    # 1. Perform transformations on the model graph
    # 2. Return the modified model wrapper
    return model
```

For example, if your custom step is named `step_transform_input` inside `transform_input.py`, you would list it in the `steps` key in `build.yaml` immediately after `step_qonnx_to_finn`:

```
steps:
  - step_qonnx_to_finn
  - transform_input.step_transform_input
  - step_tidy_up
  - step_streamline
  # ... (and the rest of the default build steps)
```

4 Task 3: PYNQ Board Evaluation

Compile your accelerator using `finn build build.yaml`. Once compilation completes, copy the generated bitstream and driver files to the PYNQ-Z2 board.

Because PyTorch and Torchvision are not installed on the board, your evaluation notebook must rely exclusively on `numpy` to preprocess test images and measure quality metrics.

4.1 Loading the Overlay

You do not need to rewrite the overlay driver. Instead, reuse the driver files generated in a previous task.

- Use the `FINNDMAOverlay` class from the driver file.
- Locate the `settings_task4.json` file inside your build output folder. Load it into your Jupyter Notebook and pass its contents directly as keyword arguments (`**driver_settings`) to initialize your overlay:

```
import json
from driver import FINNDMAOverlay

BITFILE_PATH = "./bitfiles/task4.bit"
SETTINGS_FILE = "./settings_task4.json"

with open(SETTINGS_FILE, "r") as f:
    driver_settings = json.load(f)["driver_information"]

accel = FINNDMAOverlay(
```

```
    BITFILE_PATH,
    batch_size=1000,
    **driver_settings
)
```

4.2 Batched Inference

To get maximum throughput from the DMA, you must feed the images in batches of 1000:

1. Pre-allocate a contiguous NumPy array of shape `(n_batches, 1000, 32, 32, 3)` and dtype `np.uint8`.
2. Load and copy the GTSRB test images directly from the `GTSRBDataset` class.
3. Execute the DMA using `accel.execute(input_data)` for each batch.

5 Links

- FINN+ Repository: https://github.com/eki-project/finn-plus
- FINN Transformation Passes: https://finn.readthedocs.io/en/latest/internals.html#transformation-passes
- FINN Tutorial including Pre- and Postprocessing transformations: https://github.com/Xilinx/finn/blob/main/notebooks/end2end_example/bnn-pynq/tfc_end2end_example.ipynb
- Netron Graph Visualizer: https://netron.app